

GateKeeper-Architecture-Notes

GateKeeper — Custom API Gateway with Spring Boot

Lecture 1: Project overview, architecture and initial setup

This lecture covers the design of GateKeeper and the responsibilities of its services. The initial implementation milestone is creating the Spring Boot applications and verifying them with a simple health check. The gateway features below describe the planned system; their implementation belongs to upcoming lectures.

1. Why do we need an API gateway?

An application may start with one backend. The frontend communicates with one server, and that server handles authentication, requests and business logic.

When the backend is split into services, direct access becomes harder to manage. The client may need to know where each service runs, which service handles each operation and what changes when a service moves.

Several services may also repeat the same technical work:

- Validating authentication tokens
- Checking broad access permissions
- Applying request limits
- Creating request IDs
- Recording access information
- Handling technical errors

An API gateway provides one controlled entry point in front of these backend services. The client sends requests to the gateway, which applies the relevant rules and forwards each request to its destination.

What is an API gateway?

An **API gateway is a server between API clients and backend services**. It receives a request, applies entry-point policies, forwards it to the appropriate service and returns the response.

For the client, the gateway acts as the API server. For the backend service, it acts as the caller.

The three main jobs

Job	What it means
Routing	Select the target service using the request and route configuration.
Policy enforcement	Check authentication, permissions, request limits and other configured rules.
Reverse proxying	Send the HTTP request to the target and return its response.

Routing chooses the destination. Proxying performs the exchange.

Forward proxy and reverse proxy

A forward proxy acts on behalf of clients when they contact servers. A reverse proxy acts on behalf of servers when clients contact them.

An API gateway performs reverse proxying and adds API-specific policies around it.

2. What belongs in the gateway?

The gateway handles technical entry rules. Backend services remain responsible for business rules.

For example, the gateway can require an administrator role for a protected operation. The target service must still check whether that administrator is allowed to modify a particular resource.

Moving resource ownership checks and domain logic into the gateway would couple it to every backend service. Over time, it would become another large application that must change whenever business rules change.

The gateway controls entry. The target service validates the business operation.

Related components

Component	Main responsibility	GateKeeper scope
API gateway	Route API traffic and apply entry policies.	Smart Gateway Service
Reverse proxy	Forward requests and return backend responses.	Part of the Smart Gateway
API management	Define, store and update routes and policies.	API Management Service
Load balancer	Select an instance of a service.	Excluded
Service discovery	Locate running service instances.	Excluded; target addresses are configured
Service mesh	Manage internal service-to-service traffic.	Excluded

3. What are we building?

GateKeeper is a backend-focused API gateway and API management platform built with Spring Boot, direct HTTP communication, Redis and Kafka.

The project demonstrates how gateway responsibilities work internally. It is an educational implementation with a defined scope, rather than a replacement for a mature production gateway.

Main applications

Application	Purpose
Smart Gateway Service	Handle and forward live client requests.
API Management Service	Manage users, routes and gateway policies.
Analytics Service	Process access events and provide request statistics.

Small Catalog and Order demo services will act as targets behind the gateway. Their purpose is to demonstrate forwarding and policies; they are not separate, full-sized business projects.

Planned capabilities

Area	Capabilities
Request identification	Generate or propagate a correlation ID.
Routing	Match active routes using configuration held in memory.
Security	Validate JWTs locally and enforce route-level role requirements.
Traffic control	Use Redis for rate limiting.
Duplicate request handling	Use Redis for configured POST idempotency.
Forwarding	Proxy HTTP requests with route-specific timeouts.
Transformation	Rewrite paths and add or remove selected headers.
Error handling	Return consistent gateway errors.
Analytics	Publish access events to Kafka for later processing.
Configuration	Store and version route settings; notify the gateway about changes.
Auditing	Record administrative actions using focused AOP auditing.

Dynamic routing

Dynamic routing means that routes are stored as configuration instead of being hard-coded into a large conditional block.

A route describes information such as its destination, allowed methods, authentication requirements, permitted roles, request limit, timeout and active status.

The Management Service stores these settings in MySQL. The Gateway loads active routes into memory and uses them for route matching. Updating a route does not require changing and rebuilding the Gateway's source code.

4. Data plane and control plane

Separating live traffic from configuration management is a central design decision.

Plane	Service	Responsibility
Data plane	Smart Gateway	Apply existing rules to live requests.
Control plane	API Management	Define and change those rules.

The data plane needs a short, predictable request path. The control plane handles administration and durable configuration storage.

Once the Gateway has loaded its routes, it can continue using that configuration during a temporary Management Service outage. New configuration loads and updates depend on the Management Service becoming available again.

5. Why build a custom gateway?

Using an API gateway is common and useful. Building a complete production gateway from scratch is a much larger commitment.

A production gateway may need high concurrency, connection pooling, streaming, multiple protocols, certificate management, service discovery, load balancing, resilience mechanisms, monitoring and ongoing security maintenance.

GateKeeper focuses on a smaller set of responsibilities so the underlying design is easier to understand. It brings together Spring Security, filters, interceptors, HTTP communication, Redis, Kafka, database ownership, exception handling and AOP in one project.

Features outside this project's scope

- Spring Cloud Gateway, Eureka and Spring Cloud Config
- Service discovery and load balancing
- Circuit breakers and automatic retries
- General response caching
- Request or response body transformation
- XML-to-JSON conversion
- WebSocket, gRPC and GraphQL proxying
- File uploads and streaming proxy support
- TLS termination

- API keys, billing and quota products
- Business-process orchestration

These features can be useful in other systems. They are excluded here to keep the project focused and manageable.

In particular, automatic retries can add delay and duplicate writes. Redis is used for rate limits and idempotency, not general caching of backend responses.

6. How the design controls latency

A gateway adds an extra network hop and processing work. It cannot provide zero additional latency.

GateKeeper keeps that extra work limited by avoiding unnecessary synchronous dependencies.

If the Gateway contacted separate authentication, routing, logging and analytics services for every request, each call would add time and another possible point of failure.

Instead, GateKeeper keeps routing and token validation local. Redis is consulted for policies that require shared state. The target service is then contacted over HTTP.

Work	Where it happens	Reason
Route matching	Gateway memory	Avoid a Management or database call for every request.
JWT validation	Gateway process	Verify the token using configured key material.
Role checks	Gateway process	Compare the authenticated roles with the loaded route policy.
Rate limiting	Redis	Maintain shared counters with atomic operations.
Idempotency checks	Redis	Coordinate duplicate requests using shared temporary state.
Path and header changes	Gateway process	Apply the loaded transformation rules locally.

Work	Where it happens	Reason
Analytics storage	Analytics Service	Process events outside the live response path.

Local JWT validation

The Management Service will authenticate users and issue signed JWTs. The Gateway will validate those tokens locally rather than calling the Management Service for every protected request.

This removes a recurring network call. It also means the Gateway is not fetching current user information on every request. Token expiry, key rotation and revocation are separate concerns; full revocation is outside this project's scope.

Routes in memory

MySQL remains the durable source of route configuration. The Gateway's in-memory registry is the copy used for fast request processing.

Routes are loaded at startup and refreshed when configuration changes. The Gateway does not read MySQL directly.

7. Redis: immediate shared state

Redis is a fast key-value data store. GateKeeper uses it for short-lived state needed while deciding whether a request can continue.

Why not use only Java memory?

Memory belongs to one running process. If multiple Gateway instances maintained separate counters, each instance would see only the requests it received.

Redis provides shared state that those instances can consult. Even with one Gateway running locally, this demonstrates the reason for separating shared runtime state from process memory.

Key concepts

Concept	Meaning
Key	Identifies a stored value.
Value	The counter, status or response information stored under a key.
TTL	A time limit after which the key expires automatically.
Atomic operation	A change performed as one indivisible operation.

Atomicity matters when requests arrive together. Separate reads and writes can allow two requests to act on the same old value. A Redis Lua script can group related counter and expiry operations into one atomic operation.

Rate limiting

Rate limiting restricts how many requests a user or client may make to a route within a time window.

Redis holds the counter. The Gateway updates it atomically, checks the configured limit and rejects requests that exceed it. Expiration removes counters after their useful window.

POST idempotency

Idempotency handling helps avoid repeating an operation when a client resends the same request.

For configured routes, Redis will hold temporary processing state and a saved successful response associated with an idempotency key. A matching completed request can receive the saved response without forwarding the operation again.

This is different from general response caching. It is tied to identifying a repeated operation. The detailed implementation and failure cases are reserved for later lectures.

Redis boundaries

Redis does not hold the authoritative route configuration or analytics history. MySQL and MongoDB have those responsibilities.

8. Kafka: asynchronous events

Kafka is a durable event-streaming platform. A producer writes an event to a topic, and a consumer reads it independently.

The consumer does not need to process the event immediately. Kafka retains records according to its configured retention, allowing consumers to process them later.

Key terms

Term	Meaning	GateKeeper example
Producer	Publishes events.	Gateway or Management Service
Topic	A named stream of events.	Access events or configuration events
Consumer	Reads and processes events.	Analytics Service or Gateway
Partition	An ordered subdivision of a topic.	Allows work to be distributed.
Offset	A record's position within a partition.	Tracks reading progress.
Consumer group	Consumers sharing the work of a topic.	Analytics consumers working together.

Access events

The Gateway publishes request outcome metadata to `gateway-access-events`. The Analytics Service consumes these events and stores them in MongoDB.

Useful metadata includes the event ID, correlation ID, route, request method, outcome, duration and whether a rate-limit rejection or idempotency replay occurred.

Passwords, JWTs, cookies and authorization headers must not be included in access events.

Configuration events

The Management Service publishes route-change notifications to `gateway-configuration-events`.

A notification identifies what changed. The Gateway then fetches the latest configuration from the Management Service and refreshes its in-memory registry.

The event is a notification; MySQL remains the source of truth through the Management Service.

Duplicate events

A consumer may receive an event again during failure recovery. The Analytics Service will use a unique `eventId` to avoid storing the same access event twice.

A `correlationId` serves a different purpose: it links work associated with one incoming request. An `eventId` identifies one event for deduplication.

Redis and Kafka serve different needs

Question	Redis	Kafka
What does it provide here?	Immediate shared state	Asynchronous event delivery
Typical data	Counters and idempotency state	Access and configuration events
Why does the Gateway use it?	Obtain a result needed for a policy decision	Notify another component about an outcome or change
How long is data retained?	Usually a short period controlled by TTL	According to topic retention

The Gateway does not wait for Analytics or MongoDB to finish processing an event. Publishing still has a cost, and producer failures need handling; an asynchronous send alone does not guarantee that events can never be lost.

9. Service responsibilities and ownership

Smart Gateway Service

The public entry point and data plane.

Its planned responsibilities are route matching, correlation IDs, JWT validation, role checks, rate limiting, idempotency, path and header rules, forwarding, timeouts, gateway errors and access-event publication.

It holds active routes in memory and uses Redis for temporary policy state. It loads configuration through the Management Service and contacts target services over HTTP.

It has no MySQL or MongoDB database and owns no business-domain tables.

API Management Service

The control plane for users, routes and policies.

It will manage credentials and roles, issue JWTs, store route configuration, maintain configuration versions and publish change notifications. AOP will support auditing of administrative actions.

It owns MySQL data for users, roles, routes, policy settings, versions and audit records. Other services must not directly modify its tables.

Analytics Service

Processes Gateway access events into operational information.

It will consume events, deduplicate them, store event documents and support statistics such as request counts, outcomes, average duration, slow routes, rate-limit rejections and idempotency replays.

It owns MongoDB data for access events and processed-event records. It does not change route configuration or participate in forwarding live requests.

Catalog and Order demo services

Small downstream services used to demonstrate routing, authorization, path rewriting, header propagation, timeouts and duplicate request handling.

Their business logic stays deliberately small. They remain responsible for their own business-level validation and authorization.

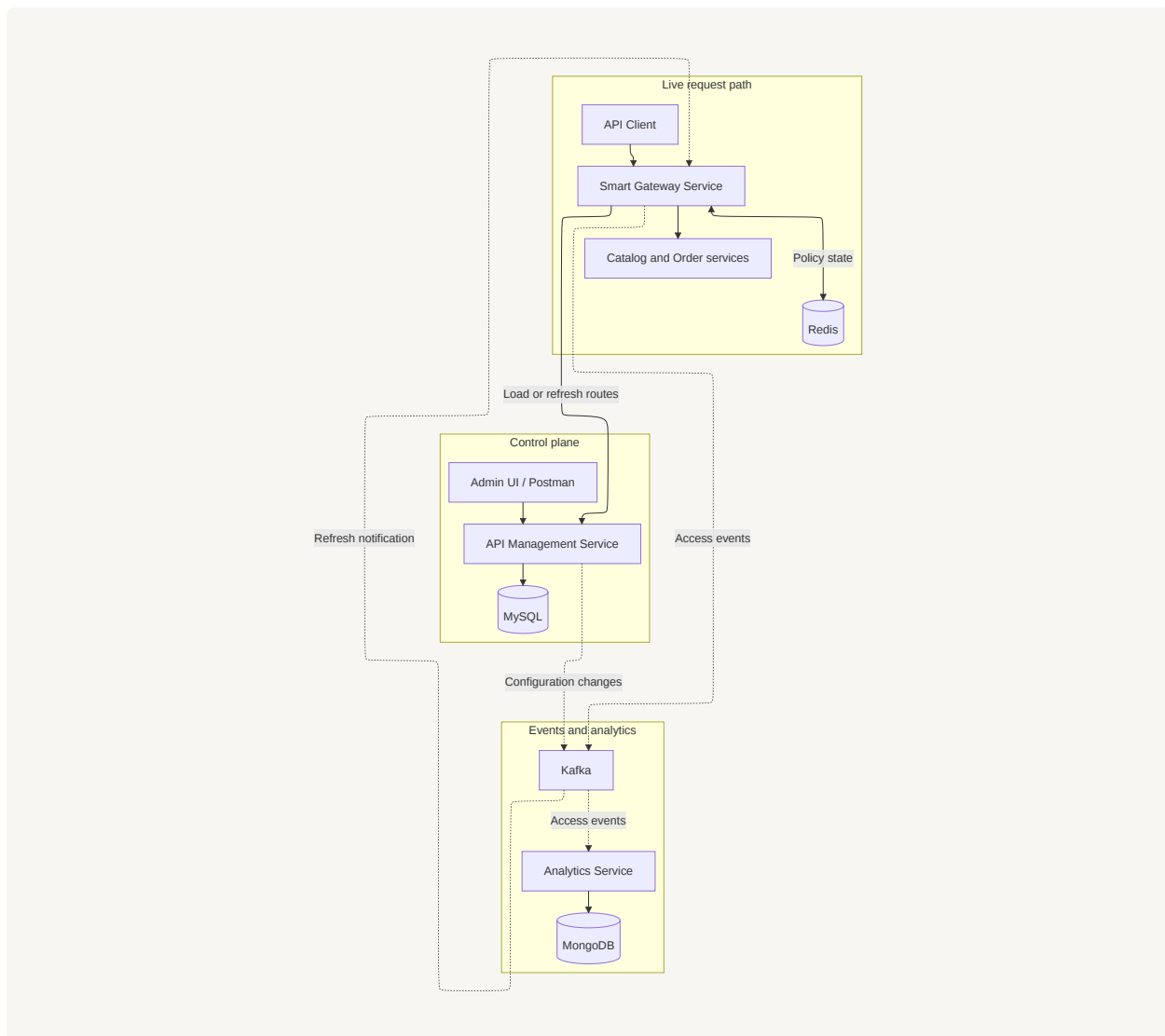
Infrastructure ownership

Component	Used or owned by	Data or purpose
MySQL	Owned by API Management	Durable users, roles, routes, versions and audits
MongoDB	Owned by Analytics	Access-event documents and processing records
Redis	Used by the Gateway's policy layer	Rate-limit counters and idempotency state

Component	Used or owned by	Data or purpose
Kafka	Shared event infrastructure	Access events and route-change notifications

10. Complete architecture

The architecture separates live traffic, configuration management and analytics. Solid arrows show direct interactions; dashed arrows show event flow.



Live request path

The client contacts the Gateway. The Gateway uses local route and security information, consults Redis when a configured policy needs it, and forwards

permitted traffic to the target service. The target's response returns through the Gateway.

Management, MySQL, Analytics and MongoDB are not contacted for every forwarded request.

Configuration path

An administrator changes configuration through the Management Service. That service saves the change in MySQL and publishes a Kafka notification. The Gateway fetches the latest configuration and updates its local registry.

This path runs during startup and configuration refresh, not for every client request.

Analytics path

The Gateway publishes an access event to Kafka. The Analytics Service consumes it and stores the information in MongoDB. The client does not wait for this processing to complete.

Architectural boundaries

- Only the Smart Gateway forwards live target API traffic.
- The Gateway accesses route configuration through Management, not through MySQL.
- The Gateway does not call Analytics or MongoDB during forwarding.
- Analytics does not administer routes.
- Each service controls its own persistent data.
- Target services retain business-specific security checks.

11. What happens when a component fails?

Component unavailable	Intended effect
API Management	Already-loaded routes remain usable; administration and refreshes pause.

Component unavailable	Intended effect
Analytics	Forwarding continues; retained Kafka events can be processed later.
MongoDB	Forwarding continues; analytics storage is affected.
Target service	Requests to that target fail through the Gateway.
Slow target	The configured route timeout limits waiting.
Redis	Routes requiring Redis-backed policy decisions fail closed in this project.
Kafka	Event publication or delivery is affected; producer failures need explicit handling.

Fail closed means refusing a request when a required policy cannot be checked, rather than silently bypassing the policy.

Separation reduces unnecessary dependencies. It does not make the system immune to failures: a component required for the current request can still affect that request.

12. Initial setup and health check

The first implementation milestone uses three Spring Boot applications:

- `smart-gateway-service`
- `api-management-service`
- `analytics-service`

Each application should start independently, use a distinct port and answer a simple health-check request.

The health check confirms that the application can start and respond over HTTP. It does not, by itself, prove that Redis, Kafka, MySQL, MongoDB or the complete gateway workflow is ready.

The supplied script does not include the health-check implementation, so no controller code, endpoint path or port values are reproduced here.

Quick revision

Concept	Remember
API gateway	One controlled entry point: routing, policies and proxying.
Data plane	Smart Gateway handles live requests.
Control plane	API Management defines and stores the rules.
In-memory routes	Avoid repeated configuration lookups during forwarding.
Local JWT validation	Avoid an authentication-service call for every token.
Redis	Shared temporary state for rate limiting and idempotency.
Kafka	Access events and configuration-change notifications.
MySQL	Durable configuration owned by Management.
MongoDB	Analytics data owned by Analytics.
Latency	The gateway adds a hop but avoids unnecessary synchronous calls.
Business logic	Remains in target services.
Current milestone	Architecture, independent application startup and a basic health check.